TECHNICAL EVALUATION · FARID OS

The Marketing Monster v1.0 — Design Review

with the evaluator's recommended build, v1.1

Document	003 — channel/TO_FARID
Subject	MIND SESSION STATE — MARKETING MONSTER v1.0
Prepared for	Farid · FARIDHD1969@aol.com
Evaluator	Cloud session, Faridunhill/Claude-Project
Date	31 July 2026
Decision needed	Ratify v1.1 · answer the Well question · release to FORGE

VERDICT

CONDITIONAL PASS — 6.6 / 10

The architecture is right. Three specification gaps — measurement, evidence rules, and the wall — must close before code is written.

ARCHITECTURAL FINDINGS ONLY · SEE §0 FOR WHAT THE EVALUATOR COULD NOT SEE

0. SCOPE AND PROVENANCE — what this evaluator could and could not see

The honesty law applies to this document about itself.

Read in full: `MIND_SESSION_STATE — MARKETING MONSTER v1.0` (the design under evaluation), and this repository — `CLAUDE.md`, the channel protocol, `docs/`, the existing encyclopedia/Professor work.

NOT seen, and therefore not assessed: the eBay sold-item export, the hub/cabinet data, GroundTruth's database, Ashcombe's operation, the current local "writes only" agent's actual code, and whatever P2 shipped. I ran on the cloud front; I do not have Farid's PC in context.

What that means for these findings. Every finding below is **architectural** — about the design's structure, contracts, failure modes and build order. Nothing here is a claim about your data, your numbers, or the quality of code that exists. Where a finding depends on a number I do not have, it says so. No estimate in this document is a measurement.

---

1. VERDICT

CONDITIONAL PASS. The architecture is right; the delivery plan and the two ends of the loop are not yet specified enough to hand to FORGE.

v1.0 makes the correct core diagnosis — P2 failed because it built a Maker and a Mouth with no Well, no Digger and no Scale, so nothing it produced could ever teach it anything. Closing that loop is the whole game, and v1.0 closes it on paper. The organ split is clean, the human-in-the-loop reserved powers are correctly drawn, and "frozen brain, living playbook" is the right architecture for a local agent — it is cheap, auditable, reversible, and it does not depend on retraining anything.

Three things block a clean handoff to code. Each is a *specification gap*, not a design error: the **measurement contract** is named but undefined; the **learning rule** has no evidence threshold, which at pipe-sale volumes will manufacture confident nonsense; and the **wall** is asserted as "structure, not promises" while being, as written, a promise. Fix those three and the build is safe to start.

One structural change I recommend beyond the blockers: **build one clone end-to-end before cloning anything.** v1.0's build order is horizontal (Scale + Well for all three, then Digger for all three…). Horizontal across three businesses is how you get three half-built loops and a repeat of P2 at triple cost.

Scorecard

#	DIMENSION	SCORE	NOTE
1	Problem diagnosis (why P2 failed)	**9 / 10**	Correct, and correctly generalized into the loop
2	Architecture & separation of concerns	**8 / 10**	Six organs are the right six; contracts between them are missing
3	Learning mechanism (frozen brain / living playbook)	**8 / 10**	Right mechanism…
4	Learning *safety* (evidence rules)	**4 / 10**	…with no threshold, no expiry, no retirement. Highest risk in the document
5	Measurement design (the Scale)	**4 / 10**	Correctly placed first in build order, but has no schema, no attribution honesty, no window
6	Data isolation (the wall)	**5 / 10**	Right principle; the leak path runs through the shared cookbook and is unaddressed
7	Legal & compliance posture	**6 / 10**	Excellent instincts (mailing-list strike, buy-don't-pirate); scraping and PII-in-the-Well unhandled
8	Build sequencing / deliverability	**5 / 10**	Horizontal across three clones by one operator; no definition of done, no kill criteria
9	Governance (who decides what)	**9 / 10**	"Monster proposes; Farid disposes" with a named reserved-powers list — exactly right
10	Intellectual honesty of the document	**9 / 10**	States what the session could not see and forbids later chats from claiming otherwise

Weighted overall: 6.6 / 10 — approve the architecture, revise the plan, then code.

---

2. WHAT v1.0 GETS RIGHT — do not let a later revision erode these

1. **The loop is the intelligence.** Well → Digger → Judge → Maker → Mouth → Scale → Well. This is the single most valuable line in the document. Every subsequent decision should be tested against "does this keep the loop closed?"
2. **Frozen brain, living playbook.** No retraining. What changes is what the agent *reads* before it writes. Human-readable, deletable, and reversible — a bad lesson is fixed by deleting one line, not by re-training a model you cannot inspect. This is the correct call and it is cheap.
3. **The edge filter.** "Does this pass through an edge we own?" — with generic lighters and handy tools cut for failing it. This is the difference between a business and a knife-fight with Amazon. The Germany consolidation route as a *structural* cost moat (8,000+ orders proven) is a real edge, not a story.

4. **Owned ground before borrowed ground.** Anything on Meta/Etsy/eBay must exist on the domain first. Combined with the standing fact that paid channels are permanently closed on the tobacco category, this is not a preference — it is the only non-revocable road, and v1.0 sees that.
5. **Reserved powers.** Category picks, floor prices, spend ceilings, anything crossing a wall — Farid alone. Correctly chosen: those are exactly the four decisions where an agent's error is expensive and silent.
6. **The channel flag as a per-clone setting, not a separate brain.** Right modelling instinct. Paid-off / paid-on is configuration; splitting the brain over it would have doubled maintenance for nothing.
7. **The document's own honesty about access.** "MIND's session had NO access… Do not let any chat claim otherwise." Make that a standing rule for every artifact in this system (see N4).

---

3. FINDINGS

Severity: **BLOCKING** = fix before FORGE writes code · **MAJOR** = fix inside the first build cycle · **MINOR** = fix when convenient, cheap now and expensive later.

B1 BLOCKING The Scale has no measurement contract, and it is scheduled first

v1.0 correctly puts the Scale in the first build wave, then never says what it records. "Records results, writes back to the Well" is not implementable. Undefined: the event schema, the identity/attribution key, the time window, and — most importantly — what counts as a *result*.

Why it breaks. The Scale is what feeds the playbook. A Scale with a loose schema does not fail loudly; it quietly writes rows that cannot be joined, cannot be cohorted, and cannot distinguish "this listing sold because of the title change" from "this listing sold because it is a 1961 Shell and those always sell." The playbook then learns from that. Garbage in the Scale is not a data-quality problem, it is a *strategy* problem one loop later.

The organic-only trap specifically. With all paid channels closed on Faridunhill, you have no click IDs and no ad-platform attribution. You will have impressions, sessions, saves, messages and sales — and for most sales you genuinely will not know what caused them. A marketing system that cannot say *"unattributable"* will invent an attribution instead. That is the honesty law being broken inside your own measurement.

Fix. Define the event schema before any other code (Appendix A). Three rules on top of it: `attribution` defaults to `unattributable` and may only be upgraded with a recorded reason; every sale row carries the cohort week and the asset version that produced it; and the Scale's weekly report must print the share of outcomes it could not attribute. If that share is 70%, the report says 70%.

B2 BLOCKING "One confirmed lesson per line" has no definition of *confirmed*

This is the highest-risk item in the document.

Why it breaks. Estate pipes are the worst possible domain for naive experiment logic: volumes are low, every item is close to unique, demand is seasonal and lumpy, and you cannot A/B test a one-of-a-kind 1961 Shell because there is only one of it. Under those conditions, "we changed the title style and the next three sold faster" is noise roughly as often as it is a lesson. Write it into the playbook and it is now permanent — *every future output obeys it*, and because

output volume is high and confirmation volume is low, the playbook fills with superstition faster than it can be corrected. A frozen brain reading a superstitious playbook is a confidently wrong marketer, and it degrades in a way that is very hard to notice from inside.

Fix — three mechanisms, all cheap (Appendix B): 1. **Two tiers.** `STRUCT` lessons ride high-volume proxy metrics (search impressions, click-through, index coverage, save rate) where N reaches the hundreds in a week and you can actually see an effect. `OUTCOME` lessons ride low-volume revenue events. Learn *aggressively* from STRUCT, *grudgingly* from OUTCOME. 2. **Every line carries its evidence and an expiry:** N, effect, birth date, review date, and the Scale query that produced it. A line that is not re-confirmed by its review date drops back to PROPOSED and stops being read. 3. **Status gate.** PROPOSED → CONFIRMED requires repetition across two non-overlapping cohorts (or an explicit Farid override, marked as such). Only CONFIRMED lines are read by the Maker. Nothing is ever confirmed on one cohort.

Note the second-order benefit: this also makes the playbook *falsifiable*, which is the same standard the encyclopedia already holds itself to. It is the marketing-side version of "wide brackets over guesses."

B3 BLOCKING The wall is described as structural but is specified as a promise

"Shared cookbook, separate kitchens. Enforced in structure, not promises." I agree with the goal and I cannot find the structure. As written, the leak path is obvious and it runs straight through the feature that makes the design attractive: **the shared cookbook.**

A lesson learned in the pipes clone — *"buyers pay a premium when the stamp is photographed at an angle that shows wear"* — is a method and travels fine. A lesson like *"pre-1970 Dunhills clear at $340+ within nine days"* is **data wearing a method's coat**, and nothing in v1.0 stops it crossing. Secondary paths: one runtime process serving several clones, one shared index or cache, one shared log file.

Fix. Structure: one filesystem root per clone, one process per clone launched with that root as its working directory, no shared index, no shared cache, no shared log. Policy: a **cookbook admission test** (Appendix C) — four written checks a lesson must pass to become shared, applied by Farid or recorded as a checklist in the commit that adds it, with the admission log kept. If a line cannot pass, it stays in its kitchen. That is a wall. What v1.0 has today is an intention.

M1 MAJOR Build vertically (one whole loop) rather than horizontally (one organ across three clones)

v1.0's build order is by organ: Scale + Well for all, then the Digger's routine, then the Judge's playbook. Three businesses × six organs = eighteen components before the first revolution of the loop completes.

Why it breaks. It reproduces P2's failure mode in a new shape — many organs, no closed loop — while tripling the surface for one operator. And the Judge's playbook, scheduled last, is the part that only becomes real once the Scale feeds it, which means the payoff arrives after eighteen components are built rather than after six.

Fix. Build **PIPES end-to-end first**: Well → Digger → Judge → Maker → Mouth → Scale → back into the Well, until one lesson has genuinely earned CONFIRMED status. Only then clone. The pipes clone earns this slot on merit — the most data, the deepest expertise, the honest-dating engine already built next door, and immediate revenue. Cloning an unproven architecture triples the debt; cloning a proven one is a copy operation.

M2 MAJOR The Judge has no decision record and no memory of what it rejected

"Worth doing? which channels? what price?" describes the Judge's questions but not its output. Without a written decision record, two things happen: the Digger re-proposes dead categories forever because nothing remembers they were killed, and there is no dataset with which to ever evaluate whether the Judge is any good.

Fix. One row per proposal (Appendix D): proposal, originating dig, which owned edge it passes through (or NONE), channel flag, effort estimate, verdict DO/DEFER/REJECT, one-line reason, review date for DEFERs. The reject log is permanent and the Digger must read it before proposing; a re-proposal must state what changed. This costs one CSV and repays it the first time it stops a repeat argument.

M3 MAJOR The Digger's quality-gap dig needs a legality and provenance layer

"Agent reads 3-star reviews / competitor stores / channels overnight" is scraping. Three exposures: platform terms (Amazon, Etsy and eBay all restrict automated collection), corpus quality (review corpora are adversarial — salted with fakes both ways), and operational fragility (rate limits and blocks make your overnight job silently return partial data, which is worse than returning nothing).

This also sits closest to the house law. "Buy, don't pirate. Honesty is the product." A marketing organ that quietly ignores platform terms is the same category of error as republishing the mirrors, and it is the one that could touch selling accounts you cannot afford to lose.

Fix. Official APIs and permitted sources first; a **source manifest** (Appendix E) recording URL, source type, fetch date, the permission basis, and which claims were derived from it. v1.0's own insight — public data is *scaffolding*, replaced plank by plank by your own transactions — becomes operational here: every scaffolding claim carries a default 180-day expiry and a `replaced_by` field pointing at the own-transaction evidence that eventually supersedes it. Store the raw evidence, not only the summary, so a claim can be re-checked when a Judge decision turns out badly.

M4 MAJOR "Intelligence only, not a mailing list" must propagate into the Well

Striking the eBay customer list as a mailing list was the right call. But the design still imports it as intelligence, and *storing* it is a separate exposure from *mailing* it. Names, emails and addresses sitting in a Well file on a workstation are a personal-data holding under both GDPR and CCPA, with retention and breach obligations attached — for a benefit you do not actually need.

Fix. Import **derived features only**: repeat-purchase counts, category affinities, price-band behaviour, recency buckets — keyed by a salted hash, never by name or email. Every strategic question the Digger will ask ("who bought twice", "which categories repeat") is answerable from derived features. Do this on day one; retrofitting it after the Well is full is a data-migration project.

M5 MAJOR GroundTruth cannot close the loop without a declared proxy conversion

v1.0 is right that free ≠ self-marketing. But free also means **no revenue event**, and the Scale's outcome metric on the other two clones is a sale. Without a substitute, GroundTruth's clone runs Well → Digger → Judge → Maker → Mouth → *nothing*. It is a Maker with extra steps — precisely the P2 condition the design exists to end.

Fix. Declare GroundTruth's proxy conversion **before** any marketing is produced, and declare exactly one primary (a verified data submission, a saved search, an account creation, or an email capture on owned ground). Everything else

is secondary. If no proxy can be named that is worth optimizing, that is a real finding: it means GroundTruth is not ready for the machine yet, and the honest move is to say so rather than to market into a void.

N1 MINOR Stamp every output with the playbook version that produced it

When a lesson is retired you need to find everything written under it. One field on each Maker output (`asset_version = playbook version hash`) turns "which of my 400 listings were written under the bad lesson?" from an archaeology project into a grep. Costs nothing now.

N2 MINOR Cap the playbook

Hard cap the CONFIRMED set (I suggest 40 lines). A cap forces retirement, keeps the file human-readable — which is the whole point of the mechanism — and keeps the prompt short enough that a local model actually weights every line. An uncapped playbook becomes a 300-line document the agent skims.

N3 MINOR Declare the cadence and the kill criteria

Nothing in v1.0 says how often the loop turns or when to stop. Recommend: one revolution per week (the Scale report is the heartbeat), one Judge cycle per week, one dig per category. Kill criterion: a category that has run three full cycles without producing a single CONFIRMED lesson or a sale is closed and written into the reject log with its reason. Systems without kill criteria accumulate zombie projects.

N4 MINOR Make the provenance line a standing rule

v1.0 does something most planning documents do not: it states what its author could not see. Make that a rule for every artifact this system produces — dig, Judge decision, Scale report, playbook line. One line at the top: what the author read, what it could not read, and what is therefore assumption. It is the same law the encyclopedia runs on, applied to marketing.

---

4. MARKETING MONSTER v1.1 — the version I would ship

Everything in v1.0 stands unless listed below. v1.1 adds contracts, evidence rules and a build order — it does not redesign the organism.

4.1 The six organs, with their contracts

ORGAN	V1.0	V1.1 ADDS
1 · THE WELL	that business's private truth, walled	Files on disk, one root per clone (§5). Personal data stored as **derived features only** (M4). A `SCHEMA.md` per Well, because a Well nobody can query is a folder
2 · THE DIGGER	inward read + outward quality-gap digs	**Source manifest** with permission basis and fetch dates; scaffolding claims carry a 180-day expiry and a `replaced_by`; must read the **reject log** before proposing (M2, M3)
3 · THE JUDGE	worth doing? channels? price? edge filter, channel flag	**Decision record** per proposal, permanent reject memory, and the playbook it reads is CONFIRMED-only (M2, B2)
4 · THE MAKER	production, only what the Judge ordered	Every output **stamped with the playbook version** that produced it (N1)
5 · THE MOUTH	owned first, borrowed second	Unchanged. The strongest rule in the document — it is what makes a permanently paid-closed category survivable
6 · THE SCALE	records results, writes back	**Event schema** (Appendix A), cohort windows, a mandatory `unattributable` bucket, and a weekly report that prints its own blind spots (B1)

4.2 The three files that are the whole system

Everything above exists to maintain three human-readable artifacts. If FORGE builds only these, the Monster is real:

1. **`well/`** + **`scale/events.jsonl`** — what happened. Append-only. Never edited, only corrected by a new row with a reason.
2. **`playbook/PLAYBOOK.md`** — what we have learned, each line carrying its evidence, its tier, its status and its expiry. Capped, deletable, read before every generation.
3. **`judge/decisions.csv`** — what we chose and what we refused, and why. The only record that can ever tell you whether the strategy organ is any good.

4.3 Build order — vertical, one clone, six waves

WAVE	BUILD	DEFINITION OF DONE
1	Scale schema + Well layout for **pipes only**	An event row can be written and queried; the Well has a `SCHEMA.md`; PII is derived-only
2	Load the pipes Well from the sold-item export	Questions in v1.0's DIG ORDER §1 are answerable by query, not by memory
3	The pipes dig (what sold fastest, at what price, which title words, who bought twice)	A written dig report with a provenance line, landed as a project file
4	First Judge cycle	≥ 5 decision rows, at least one REJECT with a reason
5	Maker + Mouth on the Judge's orders, outputs stamped	Published on owned ground first, then borrowed
6	First Scale report → first PROPOSED playbook lines	A weekly report exists, including its unattributable share
✳	**CLONE** — only now	One line has reached CONFIRMED through two cohorts

Waves 1–3 are largely *this week's work with no code*, exactly as v1.0 says. Waves 4–6 are where FORGE earns its keep. Cloning to GroundTruth and Ashcombe is a copy of a proven structure, not a new build.

4.4 Changelog v1.0 → v1.1

#	CHANGE	SOURCE FINDING
1	Build vertically (pipes whole loop) before cloning	M1
2	Scale event schema + mandatory `unattributable` bucket	B1
3	Playbook line grammar: tier, evidence N, effect, expiry, status gate	B2
4	Wall = one root + one process per clone, plus a cookbook admission test	B3
5	Judge decision record with permanent reject memory	M2
6	Digger source manifest; scaffolding claims expire and are superseded	M3
7	Well stores derived features only — no customer PII	M4
8	GroundTruth declares one primary proxy conversion before marketing	M5
9	Maker outputs stamped with playbook version	N1
10	Playbook capped at 40 CONFIRMED lines	N2
11	Weekly cadence; three-cycle kill criterion	N3
12	Provenance line on every artifact	N4

---

5. THE OPEN QUESTION — where the Hub, the Cabinet and GroundTruth's data live

v1.0 stopped here, and it is the critical path: nothing downstream can start until it is answered.

Recommendation: (b) — files on disk, dug locally, findings written to project files.

Reasons, in order of weight:

1. **The Well must be diffable, versionable and backup-able.** A file that git can diff gives you a history of corrections for free — the same property the dating cabinets already give the encyclopedia. A web UI has no history; a hosted service has someone else's history.
2. **The wall is a filesystem property.** Three roots on disk is a wall you can *see*. Three tabs in one web UI, or three folders in one connected Drive, is a wall you are trusting.
3. **No availability dependency.** A local agent that needs a browser session or a third-party service to read its own Well stops working on the day that service changes. The Well is the one organ that must never be unavailable.

4. **It matches the pattern that already works here** — the cabinets are files, the engine reads files, the encyclopedia generates from files, and `claude_EBAY_EXPORT___FIRST_READ` was produced exactly this way.
5. **A web UI is a viewer, not a store.** Option (a) is a good *later* addition on top of files — read-only, for Farid's eyes. It is a poor foundation.

Proposed layout (names are suggestions; the shape is the recommendation):

```
FARIDOS/marketing/
├── cookbook/                    # SHARED — methods only, never data
│   ├── COOKBOOK.md              # lines that passed the admission test
│   └── ADMISSION_LOG.md         # what crossed, when, who checked
└── clones/
    ├── pipes/                   # ← one root = one wall
    │   ├── well/
    │   │   ├── raw/             # exports as received, never edited
    │   │   ├── derived/         # features; NO names, NO emails
    │   │   └── SCHEMA.md
    │   ├── digger/  digs/  sources.csv
    │   ├── judge/   decisions.csv  rejects.md
    │   ├── playbook/PLAYBOOK.md
    │   ├── maker/   out/        # every file stamped with playbook version
    │   └── scale/   events.jsonl  reports/
    ├── groundtruth/             # same shape, own root
    └── ashcombe/                # same shape, own root
```

One process per clone, launched with that clone's root as its working directory. No shared index, no shared cache, no shared log. The only path between clones is `cookbook/`, and the admission test guards it.

---

6. WHAT I RECOMMEND FARID DECIDES NOW

Four decisions unblock everything; three of them are reserved powers only he holds.

1. **The Well location** — recommendation (b), files on disk, layout above. *One word unblocks waves 1–3.*
2. **Ratify or amend the twelve v1.1 changes** (§4.4). The three blockers are the ones that matter; the rest can be argued later without stopping the build.
3. **Confirm pipes as the first and only clone** until one lesson is CONFIRMED.
4. **Name GroundTruth's primary proxy conversion** — or accept that GroundTruth waits.

Then the sequence v1.0 already laid out runs: the pipes dig starts, findings land as a project file, and the build order goes to FORGE — with contracts attached, so what FORGE builds can be checked against something.

APPENDICES — the contracts, ready to hand to FORGE

Appendix A · Scale event schema (`scale/events.jsonl`, append-only)

```
{
  "ts":            "2026-08-04T14:22:00Z",
  "clone":         "pipes",
  "surface":       "site | ebay | etsy | instagram | youtube | email",
  "asset_id":      "listing/dunhill-shell-1961-a4471",
  "asset_version": "pb-2026-08-01.3",
  "event":         "impression | visit | save | inquiry | offer | sale | expired_unsold",
  "value":         340.00,
  "currency":      "USD",
  "cohort":        "2026-W32",
  "attribution":   "direct | assumed | unattributable",
  "reason":        "assumed: inquiry quoted the hub page title",
  "note":          ""
}
```

Rules. (1) `attribution` defaults to `unattributable`; upgrading it requires a `reason` string. (2) `asset_version` is the playbook version that produced the asset — this is the rollback key. (3) Rows are never edited; a correction is a new row referencing the old one. (4) The weekly report prints the unattributable share as a headline number, not a footnote.

Appendix B · Playbook line grammar (`playbook/PLAYBOOK.md`)

```
[STRUCT|OUTCOME][PROPOSED|CONFIRMED|RETIRED] <claim, one line, imperative>
  :: n=<events> :: effect=<+X% on metric> :: born=<YYYY-MM-DD>
  :: review=<YYYY-MM-DD> :: src=<scale query id | farid>
```

Example:

```
[STRUCT][CONFIRMED] Put the shape name before the finish in listing titles.
  :: n=412 impressions/2 cohorts :: effect=+18% search impressions
  :: born=2026-09-14 :: review=2027-03-14 :: src=q-titles-07
```

Promotion. STRUCT: effect visible outside the noise band in **two non-overlapping cohorts**. OUTCOME: repetition across **two independent cohorts**, never a single one. Farid may promote by decree — marked `src=farid`, which is honest rather than dressed as evidence. **Demotion.** Not re-confirmed by `review` → drops to PROPOSED and is no longer read. **Reading.** The Maker reads CONFIRMED only. The Judge may read PROPOSED. **Cap.** 40 CONFIRMED lines. Adding the 41st requires retiring one.

Appendix C · Cookbook admission test (the wall's actual mechanism)

A lesson may leave a clone and enter the shared cookbook only if **all four** are true:

1. **No business-specific proper nouns** — no customer, supplier, our-listing or our-brand names.

2. **No absolute figures from one business** — no prices, margins, volumes or counts.
3. **No customer identifier**, direct or derivable (including "the buyer who…" descriptions).
4. **It would still be true for a business selling something else.** If it would not, it is data.

Fails any one → it stays in its kitchen. Every crossing is recorded in `ADMISSION_LOG.md` with the date and who applied the test. The cookbook records the method only, never the evidence that produced it.

Worked examples. CROSSES: "Photograph the maker's mark at an angle that shows wear; buyers read wear as authenticity." DOES NOT CROSS: "Pre-1970 Dunhills clear at $340+ within nine days" — fails 1 and 2, and is a data transfer wearing a method's coat.

Appendix D · Judge decision record (`judge/decisions.csv`)

```
id, date, proposal, dig_id, edge (germany_route|audience|expertise|NONE),
channel_flag (organic_only|paid_allowed), effort_hrs, needs_farid
(floor_price|spend|category|wall|none),
verdict (DO|DEFER|REJECT), reason, review_on
```

`edge = NONE` should almost always yield REJECT — that is the edge filter doing its job, and the log is where you can later check whether it actually did. REJECTs are permanent; the Digger reads them before proposing and a re-proposal must state what changed.

Appendix E · Digger source manifest (`digger/sources.csv`)

```
id, url, source_type (api|public_page|own_transaction|interview|purchased),
fetched_at, permission_basis (official_api|terms_permit|purchased|owned),
claims_derived, expires_on (default fetch+180d), replaced_by (own-transaction evidence id)
```

Public-data claims are scaffolding: they expire on schedule and are replaced plank by plank by own-transaction evidence, exactly as v1.0 describes — this table is what makes it happen instead of being remembered.

---

7. CLOSING NOTE

The strongest sentence in v1.0 is *"the loop IS the intelligence."* It is right, and it is why the document deserves a careful evaluation rather than a rubber stamp. Everything I have added is in service of one thing: making sure that when the loop turns, what it carries back is **true**. A loop that circulates noise is not intelligence — it is a machine for becoming confidently wrong at scale, and it gets there faster than a system with no loop at all.

The evidence rules, the attribution honesty and the wall specification are the three things that decide which kind of loop you have built. They are also, all three, cheap — a schema, a line format, and a four-question checklist. Spend the week on those and the rest of the Monster is assembly.

Prepared as a technical evaluation for Farid (FARIDHD1969@aol.com), 2026-07-31. Architectural findings only; the evaluator held no access to the eBay export, hub/cabinet, or GroundTruth data — see §0.